

HOMESERVANT

Backend Developer Guide

How the HomeServant API is built

Version of 29 September 2026. Written from the code in the HomeServant-MVP repository.

Source: `docs/guides/backend_guide.html` — rebuild with `python3 docs/guides/build.py`.

Contents

1. What the backend is and what it runs on
2. Folder and module layout
3. How a request is handled
4. Authentication and sessions
5. Roles, admin levels and permissions
6. The data model
7. Business flows (bookings, payments, marketplace, chat)
8. Real-time: the WebSocket gateway
9. Background jobs
0. Files, email and other outside services
1. Configuration (environment variables)
2. Running, migrating and deploying
3. API reference
4. Conventions for new code

1. What the backend is and what it runs on

The HomeServant API is the single source of truth for users, listings, bookings, payments, the marketplace, chat, notifications and the admin console. The Flutter app (web and mobile) is only a client of it.

Piece	Choice and why
Framework	NestJS 10 (TypeScript). Its modules, guards and dependency injection keep auth and permission rules in one place.
Database	PostgreSQL on Supabase , accessed through Prisma 5 . Prisma gives a typed schema, generated queries and versioned migrations.
Real-time	Socket.IO , through Nest's WebSocket gateway, for chat, notifications and live admin updates.
File storage	Supabase Storage (public bucket). The client uploads directly using one-time signed links.
Payments	Paystack : charges, webhooks, transfers and refunds, in an escrow model.
Email	Resend in production; the console in development.
Hosting	Render (Docker web service, <code>render.yaml</code>). Migrations run automatically at start-up.

All routes are served under the `/api` prefix, for example `https://homeservant-api-c8w7.onrender.com/api/health`.

2. Folder and module layout

Everything is in `backend/`. Each feature is a Nest *module* in `src/<name>/`, usually containing a `*.controller.ts` (HTTP routes), a `*.service.ts` (business logic), a `dto/` folder (validated request bodies) and a `*.module.ts`.

Module	Responsibility
<code>auth</code>	Signup and verification, login, Google, 2FA, JWT issuing and refresh, password reset and change, deactivate and delete, cleanup of deactivated accounts.
<code>users</code>	Own profile (<code>/users/me</code>), bank details, the <code>profileCompletedAt</code> flag (set once name, phone, address, date of birth, photo, gender, occupation and marital status are all filled in; see <code>hasCompleteProfile</code>).
<code>properties</code>	Listings: browse, create, update, delete. A delete cascades to the listing's bookings and payment records, so <code>PropertiesService.deletionBlockReason</code> refuses it while the property is occupied (an unexpired <code>MOVED_IN</code> lease, a current or upcoming <code>Shortlet</code> stay, or <code>isOccupied</code>), while a tenant's payment is held in escrow before move-in, or while a payment was started in the last two hours. Open reports also block it.
<code>bookings</code>	Rentals and shortlets: requests, payment, inspection, move-in, refund, reject, renew, tenancy agreement; the lease life-cycle job.
<code>payments</code>	The single home for escrow money movement (charge, hold, release, refund).
<code>paystack</code>	Paystack HTTP client, bank list and account-name lookup, webhook route.
<code>favorites</code> , <code>reviews</code> , <code>reports</code>	Wishlist, reviews (only from real renters), user reports worked by admins.
<code>chat</code>	Threads and messages, support chats, claim, transfer, reassign, access rules, the WebSocket gateway, online presence, support alerts and cleanup jobs.
<code>notifications</code>	Notification rows plus live pushes (the only place notifications are created).
<code>vendors</code> , <code>marketplace-products</code> , <code>marketplace-orders</code>	Shops (applications and approval), products, orders and items, shipping and tracking, auto-release job.
<code>delivery</code>	Courier integration behind an interface: console (default) or GIG Logistics (scaffolding only).
<code>storage</code>	Signed upload links; checks that a URL points to one of our own images.
<code>otp</code> , <code>mail</code>	One-time codes, and email through a pluggable provider (console or Resend).
<code>admin</code>	Everything behind the admin console, plus the one-time bootstrap route.
<code>activity-log</code>	Audit trail of sensitive actions.
<code>common</code>	Guards, decorators and the error filter shared by all modules.
<code>prisma</code> , <code>health</code>	Database client; <code>GET /api/health</code> for Render's health check.

3. How a request is handled

Set up in `src/main.ts` and `src/app.module.ts`:

1. **Helmet** (security headers) and **compression**. `trust proxy` is on, so client IPs are correct behind Render's proxy.
2. **CORS**: only the origins listed in `CORS_ORIGINS` can call the API from a browser.
3. **Rate limiting** (`ThrottlerGuard` , global): 100 requests per minute per IP by default, and stricter on sensitive routes (for example the bootstrap route allows only 5 per minute).
4. **Guards** on each route decide who may call it (section 5).
5. **ValidationPipe**: request bodies are checked against their DTO classes. Unknown fields are rejected (`whitelist` and `forbidNonWhitelisted`).
6. The controller calls a service, which reads and writes the database through Prisma.
7. **HttpExceptionFilter** turns every error into `{ statusCode, message, error }`. The app shows `message` directly, so write messages for end users.

4. Authentication and sessions

Tokens

- **Access token:** a JWT signed with `JWT_ACCESS_SECRET`, valid for 15 minutes (`JWT_ACCESS_TTL`). It's sent as `Authorization: Bearer ...`.
- **Refresh token:** a JWT signed with `JWT_REFRESH_SECRET`, valid for 30 days (`JWT_REFRESH_TTL`). Only a hash of it is stored in `RefreshToken`. **Each one can be used once:** `POST /auth/refresh` revokes it and issues a new pair.
- Logging out, resetting or changing a password, deactivating or deleting all revoke refresh tokens.
- Changing either secret signs everyone out immediately. The Admin Guide's emergency procedure relies on this.

Sign-up and login flows

- **Signup:** `POST /auth/signup` creates an unverified user and emails a code; `POST /auth/verify-signup` verifies it and returns tokens. Signing up again with an email that was never verified takes over the unverified record and sends a fresh code.
- **Emails ignore case:** every email the API receives is trimmed and lower-cased (`@NormalizeEmail()` on each DTO, and Google's email too), so `Ada@example.com` and `ada@example.com` are one account. Migration `20260928140000_lowercase_emails` converted existing rows, skipping any two accounts that differ only by case (the migration file shows how to find them). Migration `20260928150000_email_lowercase_check` then added database rules (`CHECK (email = lower(trim(email)))` on `User` and `PendingAdmin`) so, together with the unique index, case-variant duplicates can't be stored even by editing the database directly. Write emails lower-cased if you ever insert rows by hand.
- **Codes:** 4 digits, valid 10 minutes. Any of the three most recent unexpired codes is accepted, so a resend doesn't invalidate the email the person already has; using one consumes them all. Five wrong guesses lock out until a new code is requested. Sending through Resend retries brief failures (rate limit, 5xx) and otherwise fails with "We couldn't send your code just now. Please try again in a minute." Sign-up codes, unread-message emails and admin alerts share one Resend account and its quota.
- **Sign-in pages:** the client sends `portal : TENANT` or `LANDLORD` from those sign-in pages, `ADMIN` from the admin console (`APP` from older app builds = any non-admin). Right after the password check, `assertPortalAllows` refuses any other kind of account with a message naming the right page (e.g. "This is a landlord account. Please sign in on the landlord page."), so nobody is signed into a dashboard from the wrong page. Google sign-in sends the page too: an existing account of another role is refused before it's linked; a new account takes the page's role. Google always refuses admin accounts and never links Google to one, since it would skip their password, 2FA and the admin sign-in log.
- **Login:** `POST /auth/login` can return:
 - tokens;
 - `requiresTwoFactor` (a code was emailed; finish with `/auth/verify-2fa`);
 - `requiresReactivation` (the account is deactivated; send the same request again with `reactivate: true`);
 - `requiresEmailVerification` (the email was never verified; a fresh signup code was sent).

- **Google:** POST /auth/google verifies Google's ID token against GOOGLE_CLIENT_ID , then finds or creates the user. An existing email is linked to the Google account.
- **Profile completion:** User.profileCompletedAt is set the first time a profile has every field signup requires, and is never cleared. The app's "finish your profile" redirect relies only on this flag, which is included in every auth response.
- **Password reset** is by emailed code (/auth/forgot-password then /auth/reset-password). It's refused for admin accounts: another admin must reset those.

5. Roles, admin levels and permissions

`User.role` is one of `TENANT`, `LANDLORD`, `VENDOR` or `ADMIN`. Admins also have `adminLevel`: `SUPPORT` < `MODERATOR` < `SUPER_ADMIN`.

Guard / decorator	What it does
<code>JwtAuthGuard</code>	Requires a valid access token and sets <code>request.user</code> .
<code>RolesGuard</code> + <code>@Roles(...)</code>	Requires one of the listed roles. For admin routes it re-reads role and level from the database on every request , so demoting or removing an admin takes effect on their very next request.
<code>AdminLevelGuard</code> + <code>@MinAdminLevel(...)</code>	Requires at least that admin level.
<code>MustChangePasswordGuard</code>	Blocks an admin still on a temporary password (except routes marked <code>@AllowMustChangePassword</code>).

Some rules are more specific than a minimum level, so services check them themselves:

- **Admin password resets:** you can't reset the password of an admin ranked above you.
- **Last super admin:** the last super admin can't be demoted or removed.
- **Support chat access** (`chat/support-access.ts` and `ChatService.canWrite / assertCanRead`):
 - non-participant admins can reach a support thread only while it's unclaimed;
 - a `SUPER_ADMIN` can read, but not write to, threads another admin is handling;
 - only a super admin can reassign.

The full admin permission table is in the Admin Guide.

6. The data model

Defined in `prisma/schema.prisma` (read its comments: most fields explain why they exist). Main tables:

Model	What it stores
User	Every account: role, admin level, profile fields (first/last/full name; a tenant's booking profile <code>bio</code> and <code>hobbies</code>), bank details, 2FA, verification, deactivation, a permanent ban (<code>bannedAt</code> , <code>banReason</code> , <code>bannedById</code>), <code>profileCompletedAt</code> , <code>adminOnDuty</code> , referral code.
RefreshToken , OtpCode , PendingAdmin	Sessions; one-time codes (10-minute lifetime); admin invites not yet confirmed (cancelled with <code>POST /admin/admins/cancel</code> ; once their code expires they can never be confirmed, so the Admins badge count deletes them).
Property , Favorite , Review	Listings (category, price unit, photos, <code>videoUrl</code> , occupancy, messaging toggle, <code>allowMonthlyPayment</code> , photo-change allowance <code>imageChangesUsed</code> / <code>imagesLockedUntil</code> , landlord hiding <code>hiddenByLandlordAt</code> , admin ranking <code>adminBoost</code> / <code>adminBoostUntil</code>), wishlists, reviews.
ListingPromotion	A landlord's paid "Featured" ad: Paystack reference, amount, days, status (<code>PENDING_PAYMENT</code> → <code>ACTIVE</code> , or <code>FAILED</code>) and the <code>startsAt</code> - <code>endsAt</code> it runs. Buying again while one runs queues the next period after it.
Booking , TenancyAgreement	Rentals and shortlets with their status and lease dates (and, for a monthly plan, <code>paymentPlan</code> , <code>monthlyRent</code> , <code>rentPaidThrough</code> and the reminder de-dupe fields); the agreement record.
Payment	The escrow ledger: one row per charge, for either a booking or an order item. Payout/refund bookkeeping: <code>payoutReference</code> , attempts and last error, <code>heldForVerificationAt</code> , <code>moneyLockedAt</code> , refund request and error, admin refund reason.
IdentityVerification	One per user: ID type and number, landlord certificate/document paths in the private bucket, review status and reviewer, plus the automated check against the issuing body (<code>idCheckStatus</code> , the name it holds, the provider and its reference) and <code>autoApproved</code> for a submission verified by that check rather than by an admin.
EvictionRequest	A landlord's request to end a tenancy, the tenant's response and the super admin's decision.
PlatformSettings	Single row of Platform Controls (<code>requireVerifiedLandlords</code> , <code>payUnverifiedLandlords</code> , <code>maxListingImageChanges</code> , <code>listingImageLockDays</code> , <code>featuredListingFeeNaira</code> , <code>featuredListingDays</code> , <code>promotedSlotEvery</code>).
VendorProfile , Product , MarketplaceOrder , MarketplaceOrderItem	Shops, products, orders (one payment per item).
Thread , ThreadParticipant , Message , ThreadTransferLog , SupportCallLog	Chats, including support threads (<code>isSupport</code> , <code>status</code> , <code>assignedAdminId</code> , reminder timestamps), their transfer history, and calls admins placed to the customer from a chat (admin, number, reason, time).

Notification	In-app notifications, with an optional <code>threadId</code> link.
Report , ActivityLog	User reports; the admin audit trail.

When an account is deleted, a sender's messages are kept but lose their sender (`onDelete: SetNull`); the same happens to activity-log actors and transfer records. Code that reads these must allow for a missing person.

7. Business flows

Bookings

- **Shortlet:** the tenant requests dates (`PENDING`). The landlord accepts or declines (`ACCEPTED` / `DECLINED`). The tenant pays (`PAID`), and payment is released to the landlord.
- **Rental (house, apartment, self-con):** creating the booking charges straight away (`PENDING` , then `PAID_AWAITING_INSPECTION` once paid). An inspection date is proposed and confirmed (`INSPECTION_PROPOSED` , `INSPECTION_CONFIRMED`). Moving in (`MOVED_IN`) releases the payment.

Before move-in, the tenant can get a refund (`REFUNDED`) or the landlord can reject. A renewal charges again, near the end of the lease: `GET /bookings/:id/renewal-quote` returns what it would cost now (current price, unit and lease months, the price last paid and the new end date); `POST /bookings/:id/renew` takes `{expectedAmount, expectedLeaseMonths}` from that quote and refuses with 409 if the landlord changed either since, so a tenant is never charged an amount they didn't see (both fields are optional for older clients). A renewal doesn't touch `Booking.priceSnapshot` until it's paid; `releaseRenewal` then sets it to the rent actually paid and updates the `TenancyAgreement`'s `rentAmount` , `priceUnit` and `leaseEndDate` , so the agreement always matches the current term.

Monthly plan (`Property.allowMonthlyPayment` , non-Shortlet only): `POST /bookings` with `paymentPlan: MONTHLY` snapshots `monthlyRent = ceil(yearly price / 12)` and `chargeBooking` charges one month. That first month is held and released exactly like a full payment; move-in sets `rentPaidThrough` one month on. `POST /bookings/:id/pay-month` (open from 7 days before it's due, and when overdue) charges the next month; the webhook sees a `MOVED_IN` monthly booking with months left and calls `releaseMonthlyInstallment` (straight to the landlord, `rentPaidThrough +1 month`) instead of `releaseRenewal` . Renewing requires every month paid, charges the new term's first month, and keeps `priceSnapshot` as the yearly rent.

Rent changes: `PropertiesService.update` compares the price, unit and `rentDurationMonths` before and after. If any changed, every tenant with a live booking on the listing (unpaid `PENDING` / `ACCEPTED` , paid and awaiting move-in, or a current `MOVED_IN` lease) gets one "Rent changed" notification with the old and new terms and what they mean for them. Payments, payouts and refunds always use the amount actually charged, so nothing already paid changes.

Payments (escrow)

All in `PaymentsService` . HomeServant charges the full amount through Paystack and holds it in its own balance (`PAID_HELD`). Later it transfers the recipient's share to their verified bank account (`RELEASED`) or refunds the payer (`REFUNDED`).

- The platform keeps **5%** on a release, or **0.2%** on a rental refunded before move-in. Fees are in basis points, and amounts are converted to kobo only at the Paystack boundary.
- Paystack confirms payments at `POST /api/paystack/webhook` . The signature (`x-paystack-signature`) is checked against the raw request body before anything is trusted. `charge.success` confirms a payment; `transfer.failed` / `transfer.reversed` put a payout back to owed. Set the Webhook URL in Paystack (Settings > API Keys & Webhooks) for each mode; see `backend/README.md` → *Paystack setup*.
- **Never twice:** every payout and refund runs under `withMoneyLock` (one money action per payment at a time) and asks Paystack first (`verifyTransfer` / `refundedSoFar`). Rent payouts that

fail no longer block the tenant; they wait on the admin Payouts & Refunds screen. Payouts to unverified landlords can be held (`payUnverifiedLandlords`).

Listings: ranking, featured ads, photos, hiding

- **Browse order** (`listing-ranking.ts` , used by `GET /properties` without `landlordId`): available listings first (a booked-out shortlet or occupied rental after all of them); among available ones, promoted listings (admin `adminBoost` 1-2 not expired, or a running paid ad) fill one slot in every `promotedSlotEvery` (positions 0, N, 2N...), the rest newest first; equally promoted listings rotate by a daily hash. The page is cut from that full order.
- **Featured ads** (`PromotionsService` , `/properties/:id/promotions`): quote, Paystack checkout (reference `promo_...` , metadata `purpose: FEATURED_LISTING`), and the webhook: `PaystackController` offers each `charge.success` to `PromotionsService.handleChargeSuccess` first, then to `PaymentsService` . Listings carry `featured / featuredUntil` .
- **Photo changes** (`PropertiesService.update`): changing the cover or gallery uses one of `maxListingImageChanges` ; the last locks photos for `listingImageLockDays` ; after the lock a fresh allowance starts. The update is conditional on `imageChangesUsed` so a double submit can't spend one change twice. Responses carry `imageChangesLeft` , `imageChangesAllowed` , `imageLockDays` , `imagesLockedUntil` .
- **Hiding**: `PATCH /properties/:id` with `isHidden` (refused while occupied). Hidden listings are left out of `GET /properties` except for their own landlord (the route uses `OptionalJwtAuthGuard` to know who's asking) and `POST /bookings` refuses them.
- **Video**: uploads to the `property-videos` folder only take mp4/mov/m4v/webm (and no other folder takes video); `StorageService.assertIsOwnVideo` checks the stored content type before `videoUrl` is saved.

Bans

`AdminService.banUser` (Moderator+) sets `bannedAt / banReason` , revokes every refresh token, emits `account:banned` to the user's live sockets, notifies and emails them. `assertNotBanned` (in `auth.service.ts`) refuses login, 2FA, Google sign-in and refresh with the reason (refresh checks it before "revoked", so an open app learns it was a ban). Banned landlords' listings are filtered from browse and can't be booked; a banned owner's shop is off the marketplace. `unbanUser` is Super Admin only (checked in the controller and the service).

Marketplace

- An order is split into items, one payment each. The vendor ships (a delivery item gets a tracking record), the buyer confirms receipt, and the vendor's share is released.
- If the buyer never confirms, a daily job releases the payment 7 days after shipping (or 7 days after payment for pickup items), unless there's an open report about that product.
- Shops are opened by tenant accounts, must be approved by an admin, and a suspended shop's products are hidden.

Chat and support

- Normal threads link two users, optionally about a property or an order, and are reused rather than duplicated.
- Tenant/landlord messaging is tied to payment (`ChatService.landlordTenantBlockReason`), so both sides do business on the platform. A paid booking (PAID through `MOVED_IN`) opens messaging

both ways. If there is no paid booking and a booking was refunded (the tenant's own refund, or the landlord rejecting a paid booking), messaging is closed *both ways* until the tenant pays again. With nothing paid yet, a landlord may reach out and the tenant may only answer a thread the landlord has already written in. Bookings count if they are on the thread's property, or on any of that landlord's properties for a thread not tied to a listing. The rule is checked when a thread is started and on every message; an existing thread still opens so its history stays readable, and `GET /threads/:id/summary` returns `lockedReason`, which the app shows in place of the composer. Support and marketplace order threads are not affected.

- When messaging closes on a refund, `PaymentsService` posts a `SYSTEM` message (no sender) through `ChatService.postBookingSystemMessage` so both sides see why in the chat. A tenant's own refund adds it only to an existing conversation; a landlord's rejection starts one if needed, and the tenant's "Booking rejected" notification links to that chat.
- A support thread starts with just the user. It sits in the shared queue until an admin claims it, either by opening it (`PATCH /threads/:id/claim`) or by replying, which is race-safe. It can then be transferred, reassigned by a super admin, or resolved.
- `ThreadTransferLog` records every hand-off with a `kind` (CLAIM, TRANSFER, REASSIGN) and the `actorId` who made it. `GET /threads/:id/handling-history` (handling admin or super admin) returns the timeline, first handler and current admin. Reading a thread (`PATCH /threads/:id/read`) also marks the caller's notifications about it read, and for admins refreshes the Messages badge, which counts conversations with unread messages plus unclaimed support chats.
- Support tools (`SupportToolsController` , admin only): `GET|POST /threads/:id/notes` (internal notes), `PATCH /threads/:id/triage` (topic, priority), `GET /threads/:id/customer-context` , `GET|POST|PATCH|DELETE /support/saved-replies` , and `GET /support/admins` (every active admin with level, on-duty, online and open-chat count, for transfers). Notes, triage and context need the same access as the thread: the handling admin, any admin while unclaimed, or a super admin.
- Customers only ever see an admin's **first name** (`adminPublicName` : `firstName` , else the first word of `fullName`) in threads, summaries, messages, live message events and notifications; admins see full names. Admin invites take `firstName / lastName` .
- The topic the customer picks (`POST /threads/support`) is stored on the thread, replaces an older topic on a still-open thread, and is shown to admins in the chat and in the "New support conversation" alert.
- Calls: `POST /threads/:id/calls {reason}` (handling admin, anyone while unclaimed, super admins; on a non-support thread, an admin in it) records a `SupportCallLog` row and a `SUPPORT_CUSTOMER_CALLED` activity entry, and returns the customer's number; `GET /threads/:id/calls` lists them.
- Auto-assignment (`ChatService.pickAssignee`): when a customer writes into an unclaimed support thread it goes to an on-duty, online admin with the fewest open support chats, ties broken by who was assigned longest ago; logged as `AUTO_ASSIGN` . Transfers and reassignments post a `SYSTEM` notice into the chat (`postThreadSystemMessage`).
- Support dashboard: `SupportChatStat` keeps one row per support conversation, deliberately not linked to `Thread` so it outlives the 30-day cleanup. It is written as the chat moves along (first customer message, first admin reply, current admin, transfers, resolution, rating) and was backfilled for older chats. `GET /support/metrics?days=7..90` (super admin, `SupportMetricsService`) returns live queue state, totals (median and 90th-percentile first reply, median resolution, rating), and breakdowns by topic, admin, hour, weekday and day. The customer can end their own support conversation with `PATCH /threads/:id/end` (resolved,

`closedByCustomer` in the stats, `SYSTEM` notice, handler notified). No one can send into a resolved support thread. The customer rates with `POST /threads/:id/rating` (once, after resolution); the summary's `canRate` says when. When nobody is available, a one-time `SYSTEM` notice tells the customer they're queued.

- Web Push (`PushService` , `POST|DELETE /push/subscriptions` , `GET /push/public-key`): every non-silent notification is also pushed to the user's subscribed browsers; subscriptions the push service reports gone (404/410) are deleted.
- Notifications for unclaimed threads are collapsed into one per admin (`NotificationsService.upsertThreadAlert`) and cleared for everyone else on claim.
- Admins marked Away get them silently (`User.adminOnDuty`).
- Resolved support threads are removed after 30 days.

Notifications

`NotificationsService.create` is the only place notifications are created. It stores the row and pushes `notification:new` to the user's live connection, with a transient `silent` flag when the app shouldn't play a sound or show a banner.

8. Real-time: the WebSocket gateway

`chat/chat.gateway.ts` . Clients connect with `auth: { token: <access token> }` . Each socket joins:

- `user:<id>` (its own room);
- `admins` , if it's an admin;
- `thread:<id>` , if it asks and is allowed.

Event (server to client)	When
<code>message:new</code>	A message is sent. It goes to the other participants, and to all admins while a support thread is unclaimed.
<code>message:read</code>	The other side read the thread.
<code>notification:new</code>	A notification was created or updated (may be <code>silent</code>).
<code>thread:claimed</code>	A support thread changed hands: claimed, transferred or reassigned.
<code>thread:access-revoked</code>	This socket was removed from a thread room it's no longer allowed in.
<code>admin:badges-changed</code>	An admin badge count changed.

Online presence (`PresenceService`) tracks connected sockets per user and records `lastActiveAt` .

9. Background jobs

Scheduled with `@nestjs/schedule`. If Render's free plan has the server asleep, a job runs on the first tick after it wakes. Every job is written to catch up safely.

Job	When	What
<code>SupportAlertsService</code>	Every minute	Assigns waiting chats to admins who have come online and on duty (<code>ChatService.assignWaitingQueue</code>). Unclaimed after 5 minutes: one reminder to all admins. Unclaimed after 15 (<code>SUPPORT_UNCLAIMED_ESCALATION_MINUTES</code>): an alert and email to every super admin. Customer unanswered for 10 minutes in a claimed chat: one alert to super admins.
<code>AccountCleanupService</code>	Daily, 03:00	Permanently deletes accounts deactivated more than 30 days ago (skipping ones with active leases or orders).
<code>SupportChatCleanupService</code>	Daily, 03:00	Deletes support messages older than 30 days, and emptied threads.
<code>LeaseLifecycleService</code>	Daily, 04:00	Re-lists properties whose lease ended without renewal; sends rent-expiry reminders 30, 15 and 0 days before; monthly plans: reminds the tenant 3 days before a month is due, and once overdue tells the tenant and flags the landlord (once per month due).
<code>MarketplaceAutoReleaseService</code>	Daily, 05:00	Releases held marketplace payments after 7 days, unless disputed.
<code>UnreadMessageEmailService</code>	Every 5 minutes	Emails tenants, landlords and vendors about messages unread for 15 minutes (one email per conversation per run, each message at most once via <code>Message.emailNotifiedAt</code> , nothing older than 24 hours). Admins aren't emailed; they have the console's alerts.

10. Files, email and other outside services

- **Uploads:** `POST /uploads/sign` takes a file name and folder. Only image extensions and known folders are accepted. It returns a one-time Supabase upload link and the file's public URL, stored under `<folder>/<userId>/<random>`. Any endpoint that saves an image URL first calls `StorageService.assertIsOwnImage`, which checks it's in our bucket and really an image.
- **Email and codes:** `OTP_PROVIDER` selects `console` (codes printed to the server log, for development) or `resend` (real email). `MailService` uses the same setting.
- **Email HTML:** any name, title, reason or other typed text put into an email's HTML goes through `escapeHtml` (`common/escape-html.ts`), so markup a user types (for example in their name) can't render in someone else's email. The plain-text part is sent as-is.
- **Paystack:** bank list, account-name lookup, charges, transfer recipients, transfers (and verifying a transfer by reference), refunds (and checking what was already refunded), webhook.
- **Private uploads:** identity documents go to the private bucket via `POST /verification/uploads/sign` (photo or PDF); saved paths must be in the user's own folder and are checked to really be an image or PDF. Admins see them only through 10-minute signed links. They are deleted with the account.
- **Google:** ID tokens are verified with Google's library against `GOOGLE_CLIENT_ID`.
- **Delivery:** `DELIVERY_PROVIDER` selects `console` (default) or `gig`. The GIG integration is unverified scaffolding.
- **ID checks:** `ID_CHECK_PROVIDER` selects `console` (default — checks nothing, so every submission waits for a moderator) or `prembly` (looks each ID number up with NIMC / the FRSC / INEC / Immigration through Prembly and compares the name on record with the name on the account). `IdCheckProvider.check` never throws — it runs inside signup, so a provider that is down returns `ERROR` and leaves the submission pending rather than failing the signup. A `MATCH` verifies a *tenant* outright; landlords always go to the queue. Lookups cost money, so one is only spent when the number is new or the last attempt gave no answer. Name comparison lives in `src/id-check/name-match.ts`. The live host is `https://api.prembly.com/identitypass`, authenticating on `x-api-key` alone; Prembly's older `*.myidentitypass.com` API hosts no longer answer at all, and the per-type endpoint paths are unconfirmed because every path under `/identitypass/verification/` returns 401 before routing resolves — see the *ID checks* section of `backend/README.md` before changing them.

11. Configuration (environment variables)

Set in Render (see `render.yaml`) and locally in `backend/.env` (copy `.env.example`).

Variable	Purpose
<code>DATABASE_URL</code> , <code>DIRECT_URL</code>	Supabase Postgres: the pooled connection for the app, the direct one for migrations.
<code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code> , <code>SUPABASE_STORAGE_BUCKET</code>	Storage access. The service role key must never reach the client.
<code>SUPABASE_PRIVATE_BUCKET</code>	Optional (default <code>homeservant-private</code>). Private bucket for identity documents; created on startup if missing. Files are only reachable through 10-minute signed links for moderators and super admins, and are deleted with the account.
<code>JWT_ACCESS_SECRET</code> , <code>JWT_REFRESH_SECRET</code>	Token signing secrets. Changing them signs everyone out.
<code>JWT_ACCESS_TTL</code> , <code>JWT_REFRESH_TTL</code>	Token lifetimes (default <code>15m</code> and <code>30d</code>).
<code>OTP_PROVIDER</code> , <code>RESEND_API_KEY</code> , <code>RESEND_FROM_EMAIL</code>	Email and code delivery.
<code>GOOGLE_CLIENT_ID</code>	Google web OAuth client; must match the app's.
<code>PAYSTACK_SECRET_KEY</code>	Paystack API and webhook signature checks.
<code>ADMIN_BOOTSTRAP_SECRET</code>	Unlocks the one-time "create first super admin" route. It refuses once any admin exists.
<code>CORS_ORIGINS</code>	Comma-separated web origins allowed to call the API.
<code>SUPPORT_UNCLAIMED_REMINDER_MINUTES</code> , <code>SUPPORT_REPLY_ESCALATION_MINUTES</code>	Support reminder delays (defaults <code>5</code> and <code>10</code>).
<code>DELIVERY_PROVIDER</code> , <code>GIG_API_KEY</code> , <code>GIG_API_BASE_URL</code>	Courier integration.
<code>ID_CHECK_PROVIDER</code> , <code>PREMBLY_X_API_KEY</code> (plus optional <code>PREMBLY_APP_ID</code> , <code>PREMBLY_BASE_URL</code>)	Checking ID numbers with the body that issued them.
<code>PORT</code>	HTTP port (Render uses <code>3000</code>).
<code>VAPID_PUBLIC_KEY</code> , <code>VAPID_PRIVATE_KEY</code> , <code>VAPID_SUBJECT</code>	Web Push keys (generate once with <code>npx web-push generate-vapid-keys</code>). Without them push is off; everything else works.
<code>APP_URL</code>	The web app's address for links in emails and where Paystack returns the payer after checkout (<code>callback_url</code>). Falls back to the first <code>CORS_ORIGINS</code> entry.
<code>SUPPORT_AUTO_ASSIGN</code>	Set to <code>false</code> to stop auto-assigning new support chats; they then wait in the queue.

12. Running, migrating and deploying

Local

```
cd backend
cp .env.example .env          # fill in Supabase + two random JWT secrets
npm install
npm run prisma:migrate        # create/upgrade the local schema
npm run start:dev             # http://localhost:3000/api, reloads on change
npx tsc --noEmit -p tsconfig.json # type-check
```

With `OTP_PROVIDER=console`, verification codes are printed in this terminal.

Changing the database

1. Edit `prisma/schema.prisma`, with a comment explaining any new field.
2. Run `npx prisma migrate dev --name what_changed`, or hand-write `prisma/migrations/<timestamp>_<name>/migration.sql`. Prefer additive changes (new nullable columns, or columns with defaults) so the live app keeps working during deploys.
3. Commit the migration folder with the code.

Tests

The tests in `backend/test` run the real services against a real, disposable Postgres (only sockets, email and push are faked). They cover the tenant/landlord payment and refund rules, auto-assignment, claims, transfers and handling history, notes and saved-reply permissions, read state and the Messages badge, unread-message emails, push cleanup, the Admins invite badge and listing deletion.

```
createdb hs_test
DATABASE_URL=postgresql://localhost/hs_test npx prisma migrate deploy
TEST_DATABASE_URL=postgresql://localhost/hs_test npm test
```

Every table is emptied before each test, so never point `TEST_DATABASE_URL` at a real database. GitHub Actions (`.github/workflows/ci.yml`) runs the type check, the migrations (and checks they match the schema), these tests, and the Flutter analyzer and tests on every pull request.

Resetting all chats

`backend/scripts/clear_all_chats.sql` is a one-off reset for testing. It deletes every conversation (with its messages, participants, transfer history and internal notes), chat notifications and support-dashboard rows, and leaves users, listings, bookings, payments, orders and saved replies alone. Run part 1 (a preview) and then part 2 (the delete, in one transaction) in the Supabase SQL Editor after taking a backup. Chat photos stay in Storage under `chat/` unless deleted there too.

Deploying

Render builds the Docker image. On start the container runs `npx prisma migrate deploy` (applying any new migrations) and then `node dist/main.js`. Deploy the backend **before or together with** any web release that depends on new fields.

13. API reference

All paths are under `/api`. "Auth" means a valid access token; "Admin (level)" means role ADMIN with at least that level.

Route	Access
GET <code>/health</code>	Public
POST <code>/auth/signup</code> <code>verify-signup</code> <code>login</code> <code>google</code> <code>verify-2fa</code> <code>resend-otp</code> <code>refresh</code> <code>forgot-password</code> <code>reset-password</code>	Public (rate-limited)
POST <code>/auth/logout</code> , PATCH <code>/auth/password</code> , PATCH <code>/auth/deactivate</code> , DELETE <code>/auth/me</code>	Auth
GET PATCH <code>/users/me</code> , PATCH <code>/users/me/bank-details</code>	Auth
GET <code>/properties</code> , GET <code>/properties/:id</code>	Public (a signed-in landlord listing their own also gets hidden ones)
GET <code>/properties/:id/promotions/quote</code> , POST GET <code>/properties/:id/promotions</code>	Landlord (owner). Featured ads
POST <code>/properties</code> , PATCH DELETE <code>/properties/:id</code>	Landlord (owner of the listing). A PATCH that changes the price, unit or lease length notifies the listing's tenants (see Bookings)
POST <code>/bookings</code> , GET <code>/bookings/mine landlord</code> , PATCH <code>/bookings/:id/respond</code> , POST <code>/bookings/:id/inspection/schedule</code> (landlord sets or changes the inspection date; confirms it), POST <code>/bookings/:id/pay inspection reject moved-in refund renew pay-month</code> , GET <code>/bookings/:id/renewal-quote</code> , GET <code>/bookings/:id/tenancy-agreement</code> (the booking's tenant or landlord; 404 before move-in), PATCH <code>/bookings/:id/inspection/respond</code>	Auth (tenant or landlord, checked per booking). <code>GET /bookings/landlord</code> includes the tenant's phone number only for tenants who have paid for that property, and leaves out unpaid rental checkouts (a non-Shortlet booking still <code>PENDING</code>): the landlord only sees, and is only notified about, a rental once it is paid
PATCH <code>/bookings/landlord/feed</code>	Landlord. <code>{cleared, bookingIds?}</code> : clear requests from the dashboard's Incoming Bookings feed (or restore them, for Undo); no ids = every pending Shortlet request. Sets <code>Booking.landlordFeedClearedAt</code> ; nothing is declined and the pending count is unchanged
POST <code>/evictions</code> , GET <code>/evictions/mine</code> , POST <code>/evictions/:id/cancel respond</code>	Landlord files/withdraws; tenant responds (EvictionsService)
POST <code>/verification/uploads/sign</code> , GET PATCH <code>/verification/me</code>	Tenant/landlord: private upload for ID documents (photo or PDF), save ID type and number, certificate and document paths. Status goes <code>INCOMPLETE</code> → <code>PENDING</code> once everything the role needs is in, at which point the number is checked with the issuing body — a tenant whose name matches comes straight back <code>APPROVED</code> (VerificationService)

GET /admin/verifications , /pending-count , /user/:userId , POST /admin/verifications/user/:userId/review , POST /admin/verifications/user/:userId/recheck	MODERATOR+. Lists mask the ID number and carry <code>idCheckStatus</code> ; the detail has the full number, the check result and signed file links. Reject needs a note, which the user is sent. <code>recheck</code> spends a fresh lookup (for a check that errored, or a submission older than the checks) and can verify a pending tenant on its own
GET PATCH /admin/platform-settings	SUPER_ADMIN. Platform Controls (single <code>PlatformSettings</code> row) plus impact stats. Every listing in property, tenant-booking and favorite responses carries <code>landlordVerified</code> and <code>hiddenUntilLandlordVerified</code> . Turning the switch on notifies and emails unverified landlords with listings (in the background). <code>payUnverifiedLandlords</code> (default true): when false, <code>PaymentsService.releaseOrHoldForLandlord</code> leaves an unverified landlord's move-in / shortlet / renewal payout at <code>PAID_HELD</code> with <code>heldForVerificationAt</code> set (the booking still advances); held payouts are released on verification (<code>VerificationService.review</code>) or all at once when it's switched back on. GET <code>/admin/platform-settings</code> also returns <code>heldPayouts</code> . <code>requireVerifiedLandlords</code> filters GET <code>/properties</code> (unless <code>landlordId</code> is given) and refuses POST <code>/bookings</code> on an unverified landlord's property. Listings carry <code>landlordVerified</code> ; <code>/users/me</code> carries <code>identityVerification.status</code> and, for landlords, <code>listingsHiddenUntilVerified</code>
GET /admin/payouts , /count , POST /admin/payouts/:paymentId/retry retry-refund refund-tenant	MODERATOR+. Stuck rent payouts and failed refunds. All money movement goes through <code>withMoneyLock</code> (one payout or refund per payment at a time, <code>Payment.moneyLockedAt</code>) and asks Paystack first: <code>verifyTransfer</code> before a payout (a fresh <code>-rN</code> reference only after a confirmed failure; the reference is saved before sending), <code>refundedSoFar</code> before a refund. Paystack <code>transfer.failed/reversed</code> webhooks put a payout back to owed. Tenant/landlord/admin refunds share <code>refundBooking</code> , which re-checks the booking inside the lock
GET /admin/evictions , GET /admin/evictions/pending-count , POST /admin/evictions/:id/review	SUPER_ADMIN only. Approval ends the lease today and sets the property unoccupied; approved evictions block renewal
GET /favorites , POST /favorites/:propertyId/toggle	Auth
GET /reviews , GET /reviews/mine , POST /reviews , DELETE /reviews/:id	Public read; tenants who rented the property write

POST /reports , GET /reports/mine	Auth. A listing: must have booked (any status), messaged about or saved it; an item: must have bought it. One open report per person per listing/item. The reporter is notified when it's resolved
GET /reports , GET /reports/open-count , GET /reports/:id , PATCH /reports/:id/status transfer	Admin (any)
GET POST /threads , POST /threads/support , GET /threads/:id/summary messages , POST /threads/:id/messages , PATCH /threads/:id/read	Auth (per-thread access rules). Messages take an optional replyToId (same thread; Message.replyTo , set null if the original is deleted) and come back with a replyTo quote. The thread summary's bookedProperty carries the property's details in a tenant-landlord chat once the tenant has paid. Inspection proposals, confirmations, declines and landlord-set dates, and payment confirmations, are posted into that chat as system messages; booking emails include propertyEmailDetails
GET /threads/support-queue , PATCH /threads/:id/claim resolve transfer , POST GET /threads/:id/calls	Admin (any, per-thread rules)
PATCH /threads/:id/reassign	Admin (Super Admin)
GET /notifications , GET /notifications/unread-count , PATCH /notifications/:id/read , PATCH /notifications/read-all	Auth
POST GET PATCH /vendors/me	Tenant or Vendor
GET /marketplace/products , GET /marketplace/products/:id	Public
GET /marketplace/products/mine , POST /marketplace/products , PATCH DELETE /marketplace/products/:id	Tenant or Vendor with an approved shop (owner)
POST /marketplace/orders , GET /marketplace/orders/mine vendor , PATCH /marketplace/orders/items/:itemId/status read ship , POST .../confirm-received , GET .../tracking	Auth (buyer or vendor, checked per item)
GET /paystack/banks , GET /paystack/resolve-account	Auth
POST /paystack/webhook	Paystack only (signature checked)
POST /uploads/sign	Auth
POST /admin/bootstrap	Public with x-admin-bootstrap-secret ; only while no admin exists
GET /admin/me , PATCH /admin/me/on-duty , GET /admin/stats activity-feed activity-log	Admin (any)

GET /admin/users , GET /admin/users/:id , PATCH /admin/users/:id , GET /admin/vendors , GET /admin/vendors/:id , GET /admin/properties , GET /admin/properties/:id , GET /admin/marketplace/products orders , GET /admin/marketplace/orders/:id , GET /admin/messages/attention-count	Admin (any). The user endpoints never return or change admin accounts (the list leaves them out, GET /admin/users/:id is 404 for one, and every user action refuses one); the dashboard's user counts leave them out too
PATCH /admin/users/:id/deactivate email password , DELETE /admin/users/:id , POST /admin/users/:id/send-password-reset , PATCH /admin/vendors/:id/approve reject suspend unsuspend , PATCH /admin/properties/:id/relist boost , DELETE /admin/properties/:id , PATCH /admin/users/:id/ban , DELETE /admin/marketplace/products/:id , GET /admin/admins/pending-count , POST /admin/admins/:id/reset-password/request confirm	Admin (Moderator)
GET /admin/admins , POST /admin/admins/request confirm cancel , PATCH /admin/admins/:id/level two-factor , DELETE /admin/admins/:id , PATCH /admin/users/:id/disable-2fa unban , DELETE /admin/activity-log , GET /admin/chat-log	Admin (Super Admin)

14. Conventions for new code

- **Never trust the client.** Every permission is checked in a guard or service, even if the app already hides the button.
- **Validate bodies with DTOs** (`class-validator`). The global pipe rejects unknown fields.
- **Money goes through `PaymentsService` only;** notifications go through `NotificationsService` only.
- **Race-safe state changes:** use conditional `updateMany` ("only if still unclaimed") or transactions, not read-then-write.
- **Audit sensitive admin actions** with `ActivityLogService.log` .
- **Allow for deleted people:** optional relations can be null after an account is deleted.
- **Error messages are user-facing:** the app shows them as-is.
- **Comment the why:** the existing code explains reasons, not mechanics. Keep doing that.